ADR PACKAGE

Architecture Decision Records

Seven decisions behind the governed pipeline, with the alternatives rejected and the evidence that led to each call.

Vijayalakshmi Ramu | October 2026 | github.com/Viji-qmofi/first-microservices-Viji-R-agentic-harness-cicd

THE SEVEN DECISIONS

1. **ADR-001** – The test-result step converted from LLM judgment to deterministic code (about 264x faster, no token cost).
2. **ADR-002** – A second conversion candidate (line-ending noise): readiness package only, not implemented. Status: Proposed.
3. **ADR-003** – A per-dimension floor for the quality rubric, not an aggregate score.
4. **ADR-004** – A three-layer memory layout: project, knowledge, reference.
5. **ADR-005** – MCP boundaries enforced server-side, per role, deny by default.
6. **ADR-006** – The smallest role set that maps to real responsibilities, including no tester role.
7. **ADR-007** – A governance policy built from near-miss evidence.

Each record has context, decision, rejected alternatives, evidence, consequences and open risks. Sections dated later were added after the original decision text, which is left unchanged.

ADR-001 ACCEPTED

Convert orchestrator_test result interpretation from LLM judgment to deterministic code

Context

Every orchestrated task run ends with the Orchestrator running `./mvnw test` and interpreting the raw Maven output to decide pass or fail – narratively, in the transcript's `orchestrator_test` step. `docs/step-classification.md` marks this step a strong candidate on all four signals: stable behavior across dozens of real runs in Modules 3-4 (DEV-02, HO-02, HO-04, HO-06, CAL-01-after), a Maven test result that is itself fully deterministic, a specification simple enough to write in one paragraph, and the highest run rate of any step in the whole system. The classification also names the edge case a deterministic version must preserve: Maven's output sometimes contains unrelated noise (Eureka stack traces from tests that deliberately exercise failure paths, WARN/ERROR log lines from retry-logic tests) that the Orchestrator has always correctly distinguished from a real failure by reading context, not by keying on the presence of certain words. "Did the build pass" has exactly one correct answer for any given Maven run; there is no genuine ambiguity for a language model to resolve here, only a well-defined parsing task currently being done by inference instead of code.

Decision

Replace the Orchestrator's narrative interpretation of `./mvnw test` output with a small script that runs the tests, parses the Surefire summary line for the real pass/fail/error counts, and returns a structured result – with no language model involved in deciding whether the build passed.

Alternatives considered

- Keep the Orchestrator's narrative interpretation. **Rejected.** Measured: averages ~14s/run active API time and ~$0.173/run in token cost (3-run baseline, module3_doc/calibration-log.md, 2026-09-25), on a step that runs on effectively every orchestrated task in the system – the highest-frequency step measured anywhere in this project. Confirmed conclusion-level consistency across 3 runs, though full response text was not compared for structural variance.
- Convert to deterministic code. **Preferred.** Measured: averages ~0.053s/run with zero token cost (3-run measurement, same log entry), a ~264x latency reduction and complete elimination of token cost versus the agentic baseline. Predictability confirmed byte-identical across all 3 runs via diff, not merely conclusion-consistent. 5/5 unit tests passing, including the named noisy-output edge case and a genuine-failure case. No quality regression found in isolation. The integrated end-to-end regression check was run afterward and passed (see Evidence), and this ADR was accepted on that basis.
- Have the Orchestrator run a regex check itself instead of full narrative interpretation, without a dedicated script. **Rejected.** This still routes a purely structural question through an LLM call, keeping the token cost and non-zero variance risk this conversion exists to remove, for no benefit over a real script – it is a smaller version of the same problem, not a different one.

Consequences

Measured benefits: ~264x lower latency and 100% token-cost elimination for a question ("did the tests pass") that has exactly one correct answer, plus perfectly repeatable output (confirmed via diff, not assumed) and a step fully auditable by reading under 90 lines of Python rather than a transcript. Known trade-off, unchanged from the original context: if a future testing framework changes Maven's Surefire output format, or if the Orchestrator ever needs to reason about *why* a specific test failed rather than just whether the suite passed, this narrow script would not cover that. Watched and confirmed: the script was integrated into the live workflow and four real orchestrated runs showed no regression attributable to it (see Evidence). Isolated measurement alone was not treated as sufficient grounds to accept this ADR, which is why acceptance waited for those runs.

Open risks

- If a future Maven or Surefire release changes the summary line's format, the parser would stop matching. It then returns `valid: false`, which fails safe, but it would block every run until fixed.
- The script reports whether the suite passed, not why a test failed. If the Orchestrator ever needs to reason about causes, this step does not cover it.
- `CLAUDE.md` says what to do when the script returns `valid: false`. It does not say what to do if the script itself crashes or is missing. This is a documented gap, not a decision.

Evidence

- Classification: docs/step-classification.md, orchestrator_test entry, strong candidate.
- Before-conversion measurement: module3_doc/calibration-log.md, "orchestrator_test deterministic conversion" entry, 2026-09-25.
- After-conversion measurement in isolation: same log entry – ~264x latency reduction, 100% token-cost elimination, byte-identical output confirmed via diff.
- Integrated end-to-end regression check: module3_doc/calibration-log.md, "orchestrator_test conversion, end-to-end regression check" entry, 2026-09-25 – 4 real orchestrated runs (2 holdout reruns, 2 genuine new-diff development tasks), no regression attributable to the conversion. All findings traced to pre-existing, unrelated gaps (retrieval citation fidelity, a third independent instance of DEV-02's documented gap) or measurement discipline, not to the converted step's own behavior.

ADR-002 PROPOSED

Convert line-ending-noise categorization during diff review from agent judgment to deterministic code (partial)

Context

Before every commit, the Orchestrator judges whether a modified file's diff represents real content change or only line-ending/whitespace churn, by running `git diff --ignore-all-space` and reasoning about the result. This has happened with completely consistent underlying logic 15+ times across this course (Module 1's build-check commits, Module 3's holdout reviews, the Module 4 port's repo-wide CRLF/LF noise, this module's own recent commits) – among the highest-frequency judgments in the entire system. `docs/step-classification.md`'s entry for this step marks it a weak candidate, not a strong one: the routine case (unrelated, incidental noise) is fully specifiable, but a real, already-encountered counterexample from this project's own history – `docker-entrypoint.sh`'s CRLF-to-LF fix during the Module 4 activation exercise – shows the same mechanical signal (an empty `--ignore-all-space` diff) needing different handling depending on task context a diff alone doesn't carry. That commit's entire point was a line-ending change; a rule that silently treats every line-ending-only diff as safe-to-exclude noise would have discarded the actual fix.

Decision

Do not fully automate the decision of what to do with a line-ending-only diff. Instead, build a narrower deterministic script that only categorizes and labels diffs (line-ending-only vs. contains-real-content) and surfaces that categorization explicitly in the run summary for human/Orchestrator review – never silently auto-excluding a file from a commit based on the category alone.

Alternatives considered

- Keep full agentic judgment for both categorization and the accept/exclude decision. Rejected as the sole option, though not fully – the routine categorization step (does this diff have real content, ignoring whitespace) runs the same mechanical check 15+ times with completely consistent outcomes, and repeating full LLM reasoning for a question `git diff --ignore-all-space`'s emptiness already answers mechanically wastes cost and latency on the 95% of cases where no contextual judgment is actually needed.
- Fully automate both categorization and the accept/exclude decision (treat every line-ending-only diff as automatically safe to exclude). Rejected, with concrete evidence: the docker-entrypoint.sh case is a real, already-occurred instance where this would have discarded the actual deliberate fix, not a hypothetical risk. Full automation here would have caused a real regression in this exact project's own history had it existed at the time.
- Partial conversion: deterministic categorization, human/Orchestrator judgment retained for the accept/exclude decision. **Preferred.** This captures the mechanizable part (is this diff pure whitespace) with a script, while preserving judgment specifically at the one point evidence shows it's needed (deciding what a line-ending-only categorization should mean for this particular task). No implementation, measurement, or integration is planned as part of this readiness package – this ADR stays Proposed.

Consequences

Expected benefits, if implemented: faster, cheaper categorization for the routine 95% of diffs, freeing the Orchestrator's own reasoning for the cases that genuinely need it (deciding what a line-ending-only label means for this specific task) rather than re-deriving the category from scratch every time. Known trade-off: this conversion is deliberately incomplete – it does not remove judgment from the workflow, only from the mechanical labeling step, so the token-cost/latency savings are smaller than a full conversion like orchestrator_test's. To watch if ever implemented: confirm the categorization script never silently causes a deliberate line-ending fix to be excluded without the human seeing it labeled as such first.

Open risks

- This is a readiness package only. Nothing is implemented, so none of the expected benefits are measured.
- A categorization script that is wrong in the rare deliberate-line-ending case could hide a change that mattered, as `docker-entrypoint.sh` did once. The design keeps the accept-or-exclude decision with the human or the Orchestrator for exactly this reason.
- The recommendation could change if the deliberate-fix pattern recurs, or never recurs across several calibration cycles.

Evidence

- Classification: docs/step-classification.md, "Line-ending-noise detection during diff review" entry, weak candidate.
- Real counterexample motivating the "partial" design: docker-entrypoint.sh CRLF-to-LF fix, Module 4 activation exercise (see conversation history and module3_doc/calibration-log.md's related discussion of that fix).
- Evidence still to gather before any future revision: whether the deliberate-line-ending-fix pattern recurs in future calibration cycles (see step-classification.md's pressure-test note) – a recurrence would strengthen the partial-conversion design; continued absence would strengthen the case for fuller automation given the existing unconditional human checkpoint.

ADR-003 ACCEPTED

Per-dimension floor instead of an aggregate score for the quality rubric

Context

Module 1's quality rubric (docs/rubric.md) scores an agent's build-health-check output across three graded dimensions (Build Result Accuracy, Warning and Error Coverage, Recommendation Consistency) plus one binary gate (Scope Discipline). A pass/fail threshold had to be chosen for how these dimensions combine into a single run verdict. This decision recurred and was deliberately re-applied at every later evaluation layer built in this project: the two-layer harness's rubric suite (Module 3.3, eval/rubric.json) and the governance red-team scoring all use the same per-dimension-floor principle, not a coincidence but a consistent, carried-forward design choice.

Decision

A run passes only if it scores 3 or higher ("Meets") on every graded dimension individually, and separately passes the binary Scope Discipline gate. There is no averaging or summing across dimensions – each one must independently clear its own bar.

Alternatives considered

- **Aggregate minimum score** (e.g., 9 out of 12 across the three graded dimensions). Rejected, with concrete reasoning recorded directly in docs/rubric.md at design time: an aggregate "would let a run pass by being excellent on one dimension while failing another outright – e.g., perfect recommendation consistency covering for a service whose result was reported wrong." A build health check has to be right on every dimension to be trustworthy; a single bad dimension masked by two strong ones is exactly the failure mode a health check exists to prevent, not a false positive it can afford to tolerate.
- **Binary pass/fail checklist for every dimension**, not just Scope Discipline. Rejected because it "can't distinguish a near-miss (e.g., three of four services correctly assessed) from a complete failure, and can't capture partial credit" (docs/rubric.md, 1.1 Dimensions). Kept binary only for Scope Discipline specifically, since a boundary violation (modifying source code it shouldn't have touched, running something beyond the build command) is a safety concern with no meaningful partial credit, not a quality gradient like the other three dimensions genuinely are.
- **Per-dimension floor (chosen).** Preferred because it captures genuine partial credit on quality dimensions where nuance matters (a near-miss on Build Result Accuracy is meaningfully different from a complete failure) while still requiring every dimension to independently clear its bar, so no single strong dimension can compensate for a weak one.

Consequences

This design was carried forward deliberately into every later evaluation system in this project, not reinvented each time: the Module 3.3 two-layer harness's rubric suite uses the identical per-dimension-floor principle (eval/rubric.json, no sum-based aggregate), and this consistency across modules is itself evidence the original Module 1 decision held up under real, repeated use rather than being abandoned once a harder evaluation problem appeared. Known trade-off: a per-dimension floor is stricter than an aggregate, meaning more runs will fail outright rather than scraping by on a high average – the right behavior for a quality gate, but it does mean a run with three strong dimensions and one borderline-weak one fails entirely rather than passing with a caveat. Open risk: as dimension count grows (the Module 3.3 rubric has four dimensions, more than Module 1's three), per-dimension floors become proportionally stricter in aggregate – worth revisiting if a future evaluation system needs many more than four or five dimensions, since requiring every one to independently clear its bar may become overly conservative at scale.

Open risks

- A per-dimension floor gets stricter as dimensions are added (see Consequences).
- No real run has yet separated the floor from an aggregate threshold, because the Module 1 re-runs fail under both. The design-time argument for the floor still rests on reasoning, not on a run where an aggregate would have passed a bad result.
- The floor is only as good as the dimension definitions. The Warning and Error Coverage dimension was scored 1 to 4 across re-runs that differed mainly in prompt wording, which shows its answer key has to be kept current.

Evidence

docs/rubric.md, sections 1.1 ("Alternatives considered") and 1.3 ("Notes on threshold design") – reasoning recorded directly alongside the rubric at design time, not reconstructed afterward. Carried-forward consistency: eval/rubric.json (Module 3.3), using the same per-dimension-floor principle with no aggregate.

Later evidence (2026-10-16)

This section was added after the Module 1 re-runs. The decision text above is unchanged.

- **Rejected alternative: binary pass/fail for every dimension.** Run 005 (`docs/iteration-log.md`) scored Warning and Error Coverage 2: it found three of the four answer-key warning categories. Final Run 2 scored 1: it reported "Warnings/errors: None", which was false. A binary checklist marks both runs "fail" and hides that the prompt change improved coverage. The 1-to-2 movement is visible only because the rubric keeps partial credit.
- **Rejected alternative: aggregate minimum.** On the six failing re-runs the scores were 3/2/3 or 3/1/3 (8 or 7 out of 12), so a 9-out-of-12 aggregate fails every one of them as well. These runs do not separate the two designs. We record that plainly rather than claim otherwise.
- **The floor in use.** Run 006 passed at 3/4/3 and its identical rerun failed at 3/2/3 on a false "JVM warnings: None". The floor failed the rerun even though accuracy and recommendation were both 3. That is the behavior the design is meant to produce.

ADR-004 ACCEPTED

Three-layer memory architecture, split by volatility and write permission

Context

Agents working on this repository need information to persist across sessions – recurring findings, stabilized conventions, repo-specific quirks, and historical review detail. module2_doc/memory-architecture.md documents what the workflow needs to remember and what must never persist (credentials, a single session's raw transcript, task-specific state no longer relevant once the task closes). A single undifferentiated memory store cannot serve this well: evolving task state needs to be agent-writable so the system can update itself as work happens, while stabilized project conventions need to require deliberate human review before changing, the same standard already applied to agent definition changes – one write-permission model cannot satisfy both needs at once.

Decision

Split persistent memory into three layers by volatility and write permission: Layer 1 (project memory directory, `.memory/project/`) – evolving, task-linked state, agent-writable, pruned/archived once a finding resolves or on a 90-day review for project-wide items. Layer 2 (knowledge files, `.memory/knowledge/`) – stable, human-maintained conventions and standards, agent-read-only, changed only via deliberate human-reviewed commit, reviewed only when the underlying policy actually changes rather than on a timer. Layer 3 (indexed reference, `.memory/reference/`) – large historical/archival material, agent-append-only, never rewritten, consulted on demand rather than loaded by default.

Alternatives considered

- **A single, undifferentiated memory store.** Rejected: a single store cannot simultaneously be freely agent-writable (needed for evolving task state) and require human review before any change (needed for stable conventions like the Feign-exception-handling pattern) – one write-permission model would have to compromise on one need or the other.
- **Placing the Feign-exception-handling convention in Layer 1 as mutable state, indefinitely.** This was the convention's actual original placement, not a hypothetical – it started in Layer 1 as a one-off decision for a single method (`placeOrder()`). It was deliberately *not* left there once reapplied to a second method (`viewAllProducts()`) rather than solved differently each time; module2_doc/memory-architecture.md records the real reasoning: treating a repeated pattern as mutable state risks a future session reasoning its way to a different approach and silently drifting, whereas graduating it to Layer 2 (human-governed, agent-read-only) makes changing it require the same deliberate review as any other project convention.
- **Three layers (chosen), with an explicit, evidenced graduation criterion.** Preferred specifically because it gives a concrete trigger for when something should move from Layer 1 to Layer 2 – not "whenever it feels stable," but "once a decision has actually been reapplied, not re-decided, a second time." This is a testable rule, not a judgment call made fresh each time.

Consequences

The three-layer split, plus a pre-commit hook (scripts/hooks/pre-commit) that scans .memory/ for credential patterns before every commit, gives this project a working, enforced data-classification boundary (Public/Internal/Confidential/Secret, documented in the same file) – not just a policy statement. This has held up under real, adversarial testing: Module 2 Exercise 2.4 deliberately attempted to store a fake secret in memory and the guardrail caught it. Known trade-off: a three-way split adds real complexity an agent must reason about correctly every time it writes (which layer does this belong in) – mitigated by the allocation decision table in memory-architecture.md giving concrete worked examples rather than only an abstract rule. Open risk: the graduation criterion ("reapplied, not re-decided, a second time") is itself a judgment call about what counts as the "same" pattern being reapplied versus a genuinely new, superficially similar decision – this has not yet been tested against a genuinely ambiguous case.

Open risks

- The graduation criterion ("reapplied, not re-decided, a second time") is a judgment call about what counts as the same pattern. It has not been tested against a genuinely ambiguous case.
- Layer 1 entries can still go stale between reviews. The `decision-005` incident below shows this, and `decision-auditor` exists to catch it, but it runs on demand and not on a schedule.

Evidence

module2_doc/memory-architecture.md – "Alternatives considered" section (the Layer 1-to-Layer 2 graduation story) and "Allocation decision table," both written at design time, not reconstructed afterward. Guardrail enforcement confirmed working under adversarial test: Module 2 Exercise 2.4's sensitive-data storage test, caught by the pre-commit hook before any secret reached a committed memory file.

Later evidence (2026-10-16)

This section was added after the Module 3 and Module 4 work. The decision text above is unchanged.

- **Rejected alternative: a single, undifferentiated memory store.** The case for separating mutable project state from stable conventions is reinforced by `decision-005.md`'s "Not yet committed" status. Its staleness was independently rediscovered four times, by four different agents (HO-01, HO-03, HO-05 and the Module 4 activation exercise), none of whom had the job of fixing it. `decision-auditor` was then built for that gap and corrected two related stale records (`decision-001`'s placeholder review date and `decision-004`'s build status). Mutable Layer 1 state with no review discipline drifted. The stable convention in Layer 2 did not (`module3_doc/calibration-log.md`, `module4_doc/decision-auditor-case-study.md`).
- **Pruning in practice.** On 2026-10-16 `decision-002` was archived with a documented reason, the first entry moved to the archive (`.memory/project/MEMORY_INDEX.md`).

ADR-005 ACCEPTED

MCP tool boundaries enforced server-side, not through client-side trust alone

Context

Three MCP servers mediate everything an agent does outside its own reasoning: `coursetools` (general file/search tools), `storage` (persistent project memory), and `retrieval` (vector/keyword search over a reference corpus). Early in this project (Module 3.1-3.2), tool restriction relied on Claude Code's own client-side `tools:` field per agent definition, plus a server-side role allow-list for `coursetools` only. Storage and retrieval had no server-side role check at all – `calling_role` was accepted and logged by storage's write operations but never checked against anything, and retrieval's `classification_ceiling` was a caller-supplied parameter with nothing tying it to the caller's actual role. This surfaced as two real, documented near-misses, not hypothetical risks: a `coursetools` general file tool could bypass storage/retrieval's classification logic entirely by reading `.memory/` directly (Module 3.2), and the Orchestrator successfully called storage's `update_entry` in a real run despite CLAUDE.md's own policy denying it (Module 3 Lab, HO-06).

Decision

Enforce role boundaries server-side, independently at each of the three MCP servers, with deny-by-default allow-lists – never relying on client-side tool restriction or a subagent's own good behavior as the actual security boundary. `coursetools` blocks `.memory/` from `file_read`/`file_write`/`codebase_search` entirely except narrow, role-scoped exceptions (`.memory/knowledge/` readable by all; `.memory/project/` read+write by `decision-auditor` only). `storage` runs `_authorize()` on every operation, checking `calling_role` against `mcp-servers/storage/allow-list.json` before touching the database. `retrieval` runs an equivalent check, computing the *effective* classification ceiling as the minimum of what a caller requests and what their role's own policy allows – never trusting the caller's requested ceiling as the final word.

Alternatives considered

- **Rely on client-side `tools:` restriction alone** (what each subagent's own definition says it can call). Rejected with direct evidence: this layer alone did not stop the `coursetools .memory/` bypass, since a general file tool with path access has no concept of which MCP server "should" own that data – the bypass used a tool the client-side restriction never flagged as dangerous, because reaching `.memory/` wasn't itself restricted at that layer. module3_doc's own documented finding: "a general-purpose file tool with no concept of document sensitivity can defeat a dedicated access-control system just by reaching the same data through a different path."
- **Trust the caller's self-declared role and parameters without a second server-side check** (storage/retrieval's original state). Rejected with direct evidence: the Orchestrator's `update_entry` call succeeded in a real run (HO-06) specifically because nothing checked `calling_role` against anything – it was accepted and merely logged. Retrieval's `classification_ceiling` had the identical gap: a caller could request any ceiling and receive exactly that, with no role-based cap.
- **Server-side deny-by-default allow-lists at every MCP server independently (chosen).** Preferred because it does not depend on any other layer behaving correctly – even if a client-side restriction is misconfigured or bypassed, the server itself refuses an operation outside the caller's declared role's grant. Verified directly: the exact HO-06 call, re-attempted after this fix, now returns `authorization_denied` naming the role and operation, confirmed via a real MCP call, not a simulated test.

Consequences

Three independent server-side boundaries now exist where there was effectively zero before storage/retrieval's `_authorize()` additions. Known, explicitly documented limitation, not hidden: every check still authorizes against a *self-declared* `calling_role`/`role` parameter – confirmed directly by a test where the Orchestrator successfully called `write_entry` claiming `calling_role="implementer"` and the call succeeded normally, indistinguishable from the real `implementer` subagent. This is an accepted, architecturally-forced limitation (subagents share one Orchestrator container rather than each running as a separately-identifiable process), not an oversight – documented in CLAUDE.md and docs/routing-and-tool-grant-map.md so it is never mistaken for verified identity. Open risk: a future environment where subagents genuinely run as separate processes could close this gap with real process-level identity; worth revisiting if this architecture changes.

Open risks

- Roles are self-declared strings, not verified identities. A caller that states a role it is not will have the claim accepted. This was confirmed by a direct test and applies to every grant.
- If subagents ever run as separate processes, process-level identity could close this gap.
- `coursetools` has no audit log of its own, so its denials are not recorded the way storage and retrieval denials are.

Evidence

module3_doc/calibration-log.md: the original `.memory/` bypass finding (Module 3.2) and the HO-06 near-miss (Module 3 Lab). CLAUDE.md, Storage and Retrieval Access section: the self-declaration limitation, confirmed by direct test. eval/enforcement-verification.md: Layer 2 storage and retrieval verification, including the real negative test (orchestrator denied `update_entry`), positive control (implementer's `write_entry` still works), and the retrieval ceiling-capping three-way test (denied / capped-to-empty / genuinely granted).

Update (2026-10-08): the coursetools server now fails closed

The decision above says deny-by-default at every MCP server, but `coursetools_server.py` carried a hardcoded fallback allow-list (including `orchestrator` for `file_read` and `file_write`) that it silently used whenever `roles.allowlist.json` was missing, so the claim was untrue for that server in that state. Reproduced directly: with the allow-list path pointing at a missing file, the original server started and exited 0 on the fallback. Fixed: `load_allowlist()` now raises and the server refuses to start; `eval/test_governed_files.py` guards it (no fallback list in the source, a missing file stops the server with the expected error, and the real file still starts it). Open risk reduced, not removed: roles remain self-declared strings, as above.

ADR-006 ACCEPTED

Role count and routing kept to the smallest set that maps to a real, distinct responsibility

Context

This project's orchestration graph grew incrementally: `planner`/`implementer` (Module 3.1), `spring-boot-reviewer` (Module 2, predating the MCP layer), two temporary calibration-only roles `reviewer_strict`/`reviewer_lenient` (Module 3 Lab, built specifically to force a reviewer-conflict scenario), and `decision-auditor` (Module 4.1). At each point, the question was whether a new responsibility needed its own subagent, or belonged inside an existing role's scope – including the Orchestrator's own.

Decision

Keep exactly as many subagents as there are genuinely distinct, independently-scoped responsibilities, and retire a role the moment its reason for existing ends – rather than accumulating roles indefinitely or collapsing distinct responsibilities into one for convenience. `orchestrator_test` (interpreting test results) stays part of the Orchestrator's own behavior rather than becoming a dedicated `tester` subagent. The temporary `reviewer_strict`/`reviewer_lenient` pair was deliberately retired once its calibration purpose was served, replaced by one real `reviewer` – not kept on as a permanent two-reviewer default.

Alternatives considered

- **Add a dedicated `tester` subagent for test execution/interpretation**, matching the course's own suggested role list. Rejected: the Orchestrator already independently runs and verifies tests as part of evaluating `implementer`'s result (CLAUDE.md, "never trust its own summary as verification"); a separate `tester` subagent would duplicate work the Orchestrator must do anyway to independently confirm a diff before approving it, adding a role boundary with no real independence benefit, since the Orchestrator would still need to re-verify a `tester`'s report the same way it re-verifies `implementer`'s. This is itself now a converted, deterministic step (ADR-001), reinforcing that it was never agentic judgment requiring its own role in the first place.
- **Keep `reviewer_strict`/`reviewer_lenient` as a permanent two-reviewer default**, since the infrastructure already existed after the calibration cycle. Rejected directly, with reasoning recorded at the time: their own agent definitions stated "TEMPORARY, for Module 3 Lab calibration cycle only," and running two full reviewer passes by default would double model cost for every review with no corresponding benefit once the specific calibration purpose (forcing a conflict to test the escalation policy) was served. Retired in favor of implementing the single real `reviewer` role that had been designed back in Module 3.1 but left as "standing in via the Orchestrator" until this point.
- **Current role set (chosen): `planner`, `implementer`, `reviewer`, `spring-boot-reviewer`, `decision-auditor`, plus the Orchestrator itself** – six responsibilities, each mapping to a genuinely distinct concern (deciding how to change something; making the change; independently evaluating it generically; independently evaluating it against Spring Boot-specific conventions; correcting stale project-memory records against git reality; coordinating and holding ultimate accountability).

Consequences

Routing stays simple to reason about precisely because role count stays minimal: `docs/routing-and-tool-grant-map.md` has six rows, not an ever-growing list accumulated for convenience. Retiring `reviewer_strict`/`reviewer_lenient` meant real git history work (deleting the agent files, removing their coursetools allow-list entries) rather than just leaving unused scaffolding in place – a small cost, paid once, in exchange for a routing map that reflects what the system actually does rather than its full history of experiments. Known trade-off: collapsing `orchestrator_test` into the Orchestrator's own behavior means this step is not independently swappable or scoped the way a true subagent would be – it inherits whatever tool access and context the Orchestrator session has, rather than a narrower, purpose-built grant. Open risk: as the system grows, a currently-reasonable "fold into the Orchestrator" decision (like `orchestrator_test`'s) could become harder to justify if the Orchestrator's own responsibilities keep accumulating – worth revisiting if the Orchestrator's own instructions grow substantially larger than they are today.

Open risks

- As the Orchestrator's own responsibilities accumulate, folding steps into it (as with `orchestrator_test`) may become harder to justify.
- With no tester agent, nothing does exploratory test design. At this scale the deterministic test step plus review covers it, and that trade-off is accepted.

Evidence

agents/reviewer_strict.md and agents/reviewer_lenient.md's own frontmatter (`"TEMPORARY, for Module 3 Lab calibration cycle only"`), git history showing their deletion and `reviewer`'s real implementation in the same commit (Module 4.1). ADR-001 (docs/adr/ADR-001-orchestrator-test-deterministic-conversion.md), confirming `orchestrator_test` was agentic judgment and not an independent role, now converted. CLAUDE.md, Orchestrator Instructions, "never trust its own summary as verification" – the actual, standing reason a separate `tester` role was never necessary.

Later evidence (2026-10-16)

This section was added after the Module 1 and Module 2 cost data was reviewed. The decision text above is unchanged.

- **Rejected alternative: keeping two reviewers as the default.** A single `spring-boot-reviewer` run cost $0.56 to $0.58 (`module2_doc/iteration-log.md`, Runs 002 and 003). A second default reviewer pass would add roughly that amount to every task, on a task that costs about $0.50 to $0.80 in total (`docs/iteration-log.md`, Runs 005 and 006). The extra reviewer would also have to serve a purpose, and the temporary pair's only purpose (forcing a conflict to test the escalation policy) had been served.

ADR-007 ACCEPTED

Governance policy structured around near-miss evidence, not generic best practice

Context

By Module 4.1, this project had two real, documented access-control failures already in its history (the `coursetools .memory/` bypass, and the Orchestrator's `update_entry` near-miss) before any formal governance policy document existed. A policy could have been written as a generic least-privilege template, or built specifically around what this project's own history showed actually goes wrong.

Decision

Structure `docs/governance-policy.md` so every denial traces to one of two sources, stated explicitly per role: a specific, cited near-miss from `module3_doc/calibration-log.md`, or plain least-privilege reasoning when no incident applies – never a denial justified by habit or by copying a generic template. Default every role to no access; widening requires the same evidence standard as a denial does: a stated reason, tied to the role's actual job, not asserted without justification.

Alternatives considered

- **A generic least-privilege policy template**, applied uniformly without connecting denials to this project's own history. Rejected: a template justifies every denial the same generic way ("least privilege"), which is defensible but weaker evidence than a denial that can point to a real incident – and it would have missed the chance to make the Orchestrator's `update_entry` denial carry its full weight, since that denial exists specifically *because* of a documented, real failure, not as a precaution against a hypothetical one.
- **Grant `decision-auditor` broad `.memory/` access from the start**, reasoning that its whole job is checking memory records so it should simply be able to read and write freely. Rejected – and this rejection is itself the policy's own best real example of least-privilege-with-justification-to-widen in action: `decision-auditor` v1 was scoped narrowly to storage's `update_entry` only, discovered during real use to be insufficient (plain git-tracked files like `MEMORY_INDEX.md` are a different system `update_entry` can't reach at all), and widened to v2 specifically and only to `file_read`/`file_write` on `.memory/project/` – not opened to all of `.memory/`, and not granted to any other role. The widening is documented with its exact justification (module4_doc/decision-auditor-case-study.md) and was verified against a path-traversal attempt (`project/../storage/`) before being trusted.
- **Policy structured around real evidence, narrow by default, justified widening only (chosen).** Preferred because every grant and every denial in the resulting document can be defended by pointing at something specific that actually happened, not by asserting a best practice in the abstract.

Consequences

The policy's own real history now demonstrates both halves of "least-privilege defaults, justification-to-widen" working as designed, not just stated: a role was denied broadly at first (correctly, since its full needs weren't yet known), the gap it caused was discovered through genuine use (not anticipated in advance), and the widening that followed was narrow, specific, and tested – not a blanket re-grant. Known trade-off: this means the governance policy was never complete on day one and required real iteration to reach its current state – a reviewer expecting a single, fully-specified policy document written once would instead find a document with its own documented evolution. Open risk: the policy's evidence-grounded structure depends on `module3_doc/calibration-log.md` continuing to be a reliable, honestly-maintained record – if that log's own completeness or accuracy degraded, the policy's citations would lose their grounding.

Open risks

- The policy's evidence-grounded structure depends on `module3_doc/calibration-log.md` staying a reliable, honestly maintained record. If that log degraded, the policy's citations would lose their grounding.
- The policy is checked against declared grants, not runtime behavior, and roles remain self-declared (`ADR-005`).
- The integrity checker can be edited by the same pull request it checks, so a pull request that weakens it needs human review.

Evidence

docs/governance-policy.md: orchestrator's `write_entry`/`update_entry` denials, explicitly citing the HO-06 near-miss. module4_doc/decision-auditor-case-study.md: the full v1-to-v2 widening story, including the real infrastructure bugs found and the path-traversal verification. module3_doc/calibration-log.md: the original near-miss evidence both denials and the widening trace back to.